B10 Feature engineering without leakage

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track B: Python for data science 2
B10 Feature engineering without leakage 3
B10-L1 Target leakage, features that know the answer 3
B10-L2 Fit on training data only 7
B10-L3 Time-safe features and point-in-time joins 12
Track B best-practice checklist 17
Appendix: solutions and answer keys 18

Track B: Python for data science

Write Python and run analyses that are correct, reproducible and easy to review.

B10 Feature engineering without leakage

Roadmap: Prerequisite for all months. Target leakage, contamination and time-safe features.

B10-L1 Target leakage, features that know the answer

Concept

Target leakage happens when a feature carries information that would not exist at the moment you make the prediction. The model learns the leak, scores brilliantly in validation, and fails in production, where the leak is absent.

Ask one question of every feature: **would I know this value at prediction time?**

Typical leaks in customer and survey data:

- **Consequences of the outcome.** `cancellation_reason` predicts churn perfectly, because only churners have one. `refund_amount` predicts complaints.
- **Values measured after the prediction date.** `calls_to_support_last_30d` computed on the extraction date, not the prediction date, includes calls made while the customer was leaving.
- **Proxies from the labelling process.** A `reviewed_by` column that is only filled in for fraud cases.
- **IDs and timestamps that encode order.** If churners were loaded in a later batch, a row ID predicts churn.

A quick screen catches the worst cases. Compute each feature's AUC on its own against the target. A single feature with an AUC near 1.0, or near 0.0 for an inverse relationship, deserves suspicion before celebration. With scikit-learn 1.8:

```
from sklearn.metrics import roc_auc_score
auc = roc_auc_score(y, X[col])
strength = max(auc, 1 - auc)
```

A screen does not prove leakage, and a leak spread across several features will pass it. The real defence is a **feature timeline**: for every feature, the date it becomes known, compared with the prediction date. Keep that table

in the project and review it with someone who knows how the data is produced.

When in doubt, leave the feature out and measure how much the model loses. A feature worth 0.15 AUC on its own that nobody can explain is more likely a leak than a discovery.

Why it matters

A churn model scored AUC 0.97 in validation. The top feature was `days_since_last_login`, computed on the day the training data was extracted. For churned customers, that included the weeks after they had already left. In production, scored on active customers, the model ranked customers little better than chance. The retention campaign built on it spent its budget on the wrong people for two months.

Worked example: weak code

```
features = ["tenure_months", "plan", "days_since_last_login",
  "cancellation_reason"]
model.fit(train[features], train["churned"])
print(roc_auc_score(test["churned"], model.predict_proba(test[features])
  [:, 1]))
```

1. **Line 1.** cancellation_reason exists only for churners. days_since_last_login was measured after the prediction date.
2. **Line 3.** The test set shares the leak, so the score looks superb.

Worked example: strong code

```
import pandas as pd
from sklearn.metrics import roc_auc_score

LEAK_THRESHOLD = 0.95


def leakage_screen(X: pd.DataFrame, y: pd.Series, threshold: float =
  LEAK_THRESHOLD) -> list[str]:
    """Numeric features that alone separate the classes suspiciously
  well."""
    suspects = []
    for col in X.select_dtypes("number").columns:
        known = X[col].notna()
        if known.sum() == 0 or y[known].nunique() < 2:
            continue
        auc = roc_auc_score(y[known], X.loc[known, col])
        if max(auc, 1 - auc) >= threshold:
            suspects.append(col)
```

```
    return suspects
```

1. **Line 10.** One feature at a time: a single near-perfect feature is the classic leak.
2. **Line 12.** No AUC without both classes. Skip rather than crash.
3. **Line 15.** Catches inverse relationships too.

For every feature, know when it becomes available. Screen single-feature AUCs, and treat a near-perfect one as a suspect.

Recap card

- **The one question for every feature** Would I know this value at prediction time?
- **Single-feature leakage screen** AUC of each feature alone. Flag max(auc, 1 - auc) near 1.
- **Common leaks** Consequences of the outcome, values measured after the prediction date, labelling proxies, order-encoding IDs.
- **Feature timeline** A table of when each feature becomes known, reviewed with the data owner.

Practice

Exercise 1. Multiple choice (level 1)

You predict on 1 March whether a customer will churn in March. Which feature leaks?

- a) Tenure in months on 1 March
- b) Number of support calls in February
- c) Number of support calls in March
- d) Plan type on 1 March

Why?

- r1) March calls happen after the prediction is made, and may be caused by leaving.
- r2) Support calls always leak.

Exercise 2. Predict the output (level 2)

What does this print? (scikit-learn 1.8)

```
from sklearn.metrics import roc_auc_score

churned = [0, 1, 0, 1]
days_inactive = [0.1, 0.9, 0.2, 0.8]
contract_months_left = [0.9, 0.1, 0.8, 0.2]
a = roc_auc_score(churned, days_inactive)
```

```
b = roc_auc_score(churned, contract_months_left)
print(a, max(b, 1 - b))
```

Exercise 3. Put in order (level 2)

Order the review of a proposed feature.

- Explain any high-AUC feature with the data owner, or drop it
- Drop features known only after the prediction date
- Find the date each feature value becomes known
- Write down the prediction date and what the model will see then
- Run the single-feature AUC screen on what remains

Exercise 4. Write the function (level 3)

Write `leakage_screen(X, y, threshold=0.95) -> list[str]`. For each numeric column of `X`, in column order, compute `roc_auc_score` on rows where the column is not missing. Return the names where `max(auc, 1 - auc) >= threshold`. Skip columns where the non-missing rows do not contain both classes.

Your code must pass these checks:

- reason_code separates the classes perfectly. Flag it.
- months_left separates the classes in reverse. Use max(auc, 1 - auc).
- Return only suspicious numeric columns, in column order. Skip text columns and one-class columns.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Numeric features that alone separate the classes suspiciously well.
def leakage_screen(
    X: pd.DataFrame,
    y: pd.Series,
    threshold: float = 0.95,
) -> list[str]: ...
```

Starter code:

```
import pandas as pd
from sklearn.metrics import roc_auc_score


def leakage_screen(X, y, threshold=0.95):
    return [c for c in X.columns if roc_auc_score(y, X[c]) > threshold]
```

Exercise 5. Architecture design (level 4)

A fraud model predicts at transaction time. Decide what to do with each candidate feature.

Options: Keep, Drop: known only after the prediction, Rebuild as of transaction time.

1. Transaction amount
2. Chargeback filed (yes/no)
3. Customer's transaction count, computed on the extract date
4. Analyst who reviewed the case

Sources

- roc_auc_score returns 0.0 for a perfect inverse ranking. (verified: Run on scikit-learn 1.8.0 and 1.9.1 in this repository.)

B10-L2 Fit on training data only

Concept

Every step that **learns from data** must learn from the training rows only: imputation means, scaling parameters, category lists, target encodings, feature selection, even the choice of outliers to drop. If a step sees the test rows, information about them flows into the model, and the test score becomes optimistic. This is **train/test contamination**.

A demonstration from this course's build, with scikit-learn 1.8. Take 100 rows of pure noise: 5,000 random features and a random 0/1 target. Nothing can be predicted.

- Select the 20 features most correlated with the target **on all 100 rows**, then cross-validate a logistic regression on them: accuracy **0.85**.
- Put the same selection **inside** each cross-validation fold: accuracy **0.48**, the chance level it should be.

The first model did not learn anything real. The selection step had already looked at every test fold's labels and picked the features that happened to fit them.

The rule for manual preprocessing:

```
params = fit_preprocessing(train)          # learn from train only
train_ready = apply_preprocessing(train, params)
test_ready = apply_preprocessing(test, params)   # reuse, never refit
```

Split **first**, before any step that learns. In cross-validation, that means refitting the preprocessing inside every fold. Doing that by hand is error-prone. scikit-learn's `Pipeline` does it for you (lesson B11-L2).

Steps that do **not** learn are safe on the full data: fixed recodes from a codebook, unit conversions, parsing dates. Steps that learn include anything with `fit`, anything that computes a mean, a quantile, a list of categories or a correlation with the target.

Why it matters

A propensity model imputed missing income with the median of the full dataset and scaled every feature on the full dataset before the train/test split. It then selected features by their correlation with the target, also on the full data. The validation AUC was 0.79. On the next month's customers it was 0.66. Most of the gap came from the feature selection: it had fitted the noise in the test rows.

Worked example: weak code

```
1 df["income"] = df["income"].fillna(df["income"].median())
2 df["income"] = (df["income"] - df["income"].mean()) / df["income"].std()
3 train, test = train_test_split(df, test_size=0.2, random_state=1)
```

1. **Line 1.** The median includes test rows.
2. **Line 2.** Scaling parameters learned from test rows too.
3. **Line 3.** The split comes after the learning, too late.

Worked example: strong code

```
 1 import pandas as pd
 2
 3
 1 def fit_income_prep(train: pd.DataFrame) -> dict[str, float]:
 5     """Learn median, mean and sd of income from training rows only."""
 6     filled = train["income"].fillna(train["income"].median())
 7     return {"median": float(train["income"].median()), "mean":
       float(filled.mean()), "sd": float(filled.std())}
 8
 9
10 def apply_income_prep(df: pd.DataFrame, params: dict[str, float]) ->
     pd.DataFrame:
11     """Impute and scale income with parameters learned elsewhere."""
 2     filled = df["income"].fillna(params["median"])
13     return df.assign(income=(filled - params["mean"]) / params["sd"])
```

1. **Line 4.** Learning happens in one place, on training rows.
2. **Line 12.** Applying reuses the learned values, for train and test alike.

Split first. Fit every learning step on training rows. Apply the fitted parameters to everything else.

Recap card

- **Train/test contamination** A learning step saw test rows: imputation, scaling, encoding, selection.
- **Selection on all rows, noise data** CV accuracy 0.85. Inside the folds: 0.48. The first number is a leak.
- **The manual rule** Split first, fit on train, apply the fitted parameters to train and test.
- **Safe before the split** Steps that learn nothing from the rows: fixed recodes, unit conversions, parsing.

Practice

Exercise 1. Multiple choice (level 1)

Which step is safe to run on the full dataset before the split?

- a) Filling missing income with the median
- b) Converting height from inches to centimetres
- c) Selecting the 20 features most correlated with the target
- d) Standardising features to mean 0 and sd 1

Why?

- r1) A unit conversion uses a fixed constant. It learns nothing from the rows.
- r2) Conversions happen before any modelling.

Exercise 2. Predict the output (level 2)

Pure noise: 100 rows, 5,000 random features, random labels. The 20 best features are selected on all rows, then 5-fold CV is run. Roughly what accuracy does it print? (scikit-learn 1.8, seed 0)

What you need to know.

The code uses four scikit-learn tools (version 1.8):

- `SelectKBest(f_classif, k=20).fit_transform(X, y)` scores each column against the labels `y` and returns only the `k` best-scoring columns.
- `LogisticRegression()` is a linear classifier.

- `cross_val_score(model, X, y, cv=5)` splits the rows into 5 folds. It fits on 4 folds, scores accuracy on the fifth, and repeats for each fold. It returns the 5 scores.
- `rng.normal(size=(rows, cols))` draws random numbers, and `rng.integers(0, 2, n)` draws `n` random labels, 0 or 1.

On pure noise, an honest model scores about 0.5 accuracy: a coin toss.

```
import numpy as np
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score

rng = np.random.default_rng(0)
X = rng.normal(size=(100, 5000))
y = rng.integers(0, 2, 100)
X_sel = SelectKBest(f_classif, k=20).fit_transform(X, y)
print(round(cross_val_score(LogisticRegression(), X_sel, y, cv=5).mean(),
    2))
```

Exercise 3. Fill the gap (level 2)

Fit the scaler on training rows, then apply it to both sets.

```
scaler = StandardScaler().fit(_____(1))
X_train_s = scaler.transform(X_train)
X_test_s = scaler._____(2)(X_test)
```

Exercise 4. Write the function (level 3)

Write `fit_prep(train) -> dict[str, float]` and `apply_prep(df, params) -> pd.DataFrame` for the column `income`.

- `fit_prep` learns `median` of income, then fills missing values with it and learns `mean` and `sd` of the filled column. Training rows only.
- `apply_prep` fills missing income with `params["median"]` and returns a new frame with `income` scaled as `(filled - mean) / sd`. If `sd` is 0, set scaled income to 0.0.

Your code must pass these checks:

- Learn the median, then mean and sd of income after filling with that median.
- Test rows must use the training median, mean and sd, not their own.
- When sd is 0, scaled income must be 0.0.
- The input frame changed. Return a new one.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Learn imputation and scaling parameters for income from training rows.
def fit_prep(train: pd.DataFrame) -> dict[str, float]: ...

# Impute and scale income with parameters learned from training rows.
def apply_prep(
    df: pd.DataFrame,
    params: dict[str, float],
) -> pd.DataFrame: ...
```

Starter code:

```python
import pandas as pd


def fit_prep(train):
    return {}


def apply_prep(df, params):
    filled = df["income"].fillna(df["income"].median())
    return df.assign(income=(filled - filled.mean()) / filled.std())
```

Exercise 5. Spot the bug (level 4)

Target encoding replaces each region with its churn rate. The model's test AUC is far above its production AUC. Tap the line that leaks, and fix the function so the rates come from training rows only.

What you need to know.

`pd.concat([a, b])` stacks DataFrames on top of each other: the rows of `a`, then the rows of `b`.

`series.map(lookup)` replaces each value with the matching entry of another Series, matched by its index:

```python
rates = pd.Series({"North": 0.1, "South": 0.3})
pd.Series(["North", "South"]).map(rates)   # 0.1, 0.3
```

```python
import pandas as pd


def encode_region(train: pd.DataFrame, test: pd.DataFrame) ->
    tuple[pd.DataFrame, pd.DataFrame]:
     """Replace region with its churn rate."""
     both = pd.concat([train, test])
     rates = both.groupby("region")["churned"].mean()
     return train.assign(region=train["region"].map(rates)),
    test.assign(region=test["region"].map(rates))
```

Write down the line number of the bug. Then write the fix.

Sources

- SelectKBest(k=20) on all rows of 100 x 5,000 noise gives 0.85 CV accuracy; inside a Pipeline, 0.48. (verified: Run on scikit-learn 1.8.0 and 1.9.1 with NumPy 2.4.6 in this repository (seed 0).)
- scikit-learn provides TargetEncoder with internal cross fitting. (verified: scikit-learn 1.8 API reference; sklearn.preprocessing.TargetEncoder imported in this repository.)

B10-L3 Time-safe features and point-in-time joins

Concept

When rows have dates, every feature must be computed **as of** the row's own date, using only earlier data. Three tools in pandas 3.0 cover most cases.

Prior-only aggregates. "Total spend before this order" must exclude the current order. Take the running total and subtract the current value, or shift first:

```
orders = orders.sort_values(["customer", "ordered_at"])
prior = orders.groupby("customer")["spend"].cumsum() - orders["spend"]
```

Point-in-time joins. To attach the latest satisfaction score **before** each prediction date, use `pd.merge_asof`. It matches each left row with the last right row whose key is at or before it, within the same `by` group. Both frames must be sorted on the key.

```
pd.merge_asof(preds.sort_values("as_of"), scores.sort_values("scored_at"),
              left_on="as_of", right_on="scored_at", by="customer",
              allow_exact_matches=False)
```

The default `allow_exact_matches=True` also matches a score recorded **at the same timestamp** as the prediction. If that score could only be known after the prediction was made, set it to `False`. A plain `merge` on customer followed by "take the latest" silently uses scores from the future.

Time-based splits. Random splits put next month's rows in training and last month's in test. For anything you will use forward in time, split by date: train on data up to a cut-off, test on data after it. scikit-learn's `TimeSeriesSplit` gives expanding windows for cross-validation.

Build a small **as-of test** for every time feature: a customer with events before and after the prediction date, and an assertion that only the earlier events count. It catches most mistakes in a few lines.

Why it matters

A next-purchase model used "customer's total orders" as a feature, computed once over the full order table. For each training row, the count included orders placed after the row's date, including the very purchase the model was trying to predict. Validation AUC was 0.91. In production, where the future does not exist yet, it was 0.68.

Recomputing the count as prior orders only brought validation down to 0.70. That was the model's true performance, and the team finally had an honest number to improve.

Worked example: weak code

```
totals = orders.groupby("customer")["spend"].sum().rename("total_spend")
rows = orders.merge(totals, on="customer")
train, test = train_test_split(rows, test_size=0.2)
```

1. **Line 1.** One total per customer across all time: every row sees its own future.
2. **Line 3.** Random split: training rows come from after test rows.

Worked example: strong code

```
import pandas as pd


def add_prior_spend(orders: pd.DataFrame) -> pd.DataFrame:
    """Spend and order count before each order, per customer. Grain: one row per order."""
    o = orders.sort_values(["customer", "ordered_at"])
    grp = o.groupby("customer")["spend"]
    return o.assign(
        prior_spend=grp.cumsum() - o["spend"],
        prior_orders=o.groupby("customer").cumcount(),
    )


def time_split(df: pd.DataFrame, date_col: str, cutoff: pd.Timestamp) -> tuple[pd.DataFrame, pd.DataFrame]:
    """Train on rows before cutoff, test on rows at or after it."""
    before = df[date_col] < cutoff
    return df[before], df[~before]
```

1. **Line 9.** Running total minus the current order: only earlier orders count.
2. **Line 10.** cumcount starts at 0, so it counts earlier orders only.
3. **Line 16.** A time split: the test set is the future, as in production.

Compute every feature as of the row's date, join with merge_asof, and split by time.

Recap card

- **Prior-only running total** groupby(key)[col].cumsum() - col, after sorting by key and date.
- **Point-in-time join in pandas** pd.merge_asof(left, right, left_on, right_on, by=..., allow_exact_matches=False), both sorted.
- **merge_asof default for equal timestamps** allow_exact_matches=True: same-time rows match. Set False when they would not be known yet.
- **Split for forward-looking models** By time: train before a cut-off, test after. TimeSeriesSplit for CV.

Practice

Exercise 1. Multiple choice (level 1)

A model will score customers each month, going forward. How should you split the historical data for validation?

- a) Random 80/20 split
- b) Train on months up to a cut-off, test on later months
- c) Test on the earliest months, train on later ones
- d) No split: use training accuracy

Why?

- r1) It matches production, where the model predicts a future it has not seen.
- r2) Recent data is always more accurate.

Exercise 2. Predict the output (level 2)

What does this print? (pandas 3.0)

What you need to know.

`pd.to_datetime([...])` turns date strings such as `"2026-01-31"` into timestamps, so pandas compares and sorts them as dates, not as text.

```
import pandas as pd

preds = pd.DataFrame({"customer": ["c1"], "as_of":
    pd.to_datetime(["2026-04-01"])})
scores = pd.DataFrame({
    "customer": ["c1", "c1"],
    "scored_at": pd.to_datetime(["2026-02-15", "2026-04-01"]),
    "nps": [3, 9],
})
```

```
out = pd.merge_asof(preds, scores, left_on="as_of", right_on="scored_at",
    by="customer")
print(out["nps"].iloc[0])
```

Exercise 3. Fill the gap (level 2)

Attach the latest earlier score to each prediction row, excluding same-timestamp scores.

```
pd._____(1)(preds, scores, left_on="as_of", right_on="scored_at",
    by="customer", allow_exact_matches=_____(2))
```

Exercise 4. Write the function (level 3)

Write `add_prior_spend(orders) -> pd.DataFrame`. `orders` has `customer`, `ordered_at` and `spend`, in any row order. Return the orders sorted by `customer` then `ordered_at`, with two new columns:

- `prior_spend`: total spend by the same customer on **earlier** orders (0 for the first).
- `prior_orders`: number of earlier orders by the same customer (0 for the first).

Your code must pass these checks:

- prior_spend must sum earlier orders only, per customer, in date order.
- prior_orders must count earlier orders only, starting at 0.
- The input frame changed. Return a new one.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Spend and order count before each order, per customer.
def add_prior_spend(orders: pd.DataFrame) -> pd.DataFrame: ...
```

Starter code:

```
import pandas as pd


def add_prior_spend(orders):
    totals = orders.groupby("customer")["spend"].transform("sum")
    return orders.assign(prior_spend=totals)
```

Exercise 5. Spot the bug (level 4)

For each prediction, the feature should be the customer's most recent NPS score strictly **before** `as_of`. Validation looks too good. Tap the line that leaks and fix it.

What you need to know.

`df.groupby(keys).tail(1)` keeps the last row of each group, in the frame's current order. After `sort_values("date")`, that is the latest row per group:

```
df = pd.DataFrame({"k": ["a", "a", "b"], "v": [1, 2, 3]})
df.groupby("k").tail(1)   # rows (a, 2) and (b, 3)
```

```
import pandas as pd


def latest_nps(preds: pd.DataFrame, scores: pd.DataFrame) -> pd.DataFrame:
    """Most recent NPS strictly before each prediction date."""
    joined = preds.merge(scores, on="customer")
    joined = joined.sort_values("scored_at").groupby(["customer", "as_of"]).tail(1)
    return joined[["customer", "as_of", "nps"]].sort_values(["customer", "as_of"]).reset_index(drop=True)
```

Write down the line number of the bug. Then write the fix.

Sources

- pd.merge_asof defaults to direction="backward" and allow_exact_matches=True. (verified: pandas API reference; run on pandas 3.0.6 in this repository.)

Track B best-practice checklist

☐ The project has a pyproject.toml, a virtual environment and pinned dependencies.

☐ ruff and mypy pass before every commit.

☐ Functions are pure where possible. Side effects sit at the edges.

☐ Dtypes are explicit. pandas code is vectorised, with no row-wise apply on large frames.

☐ Inputs are validated with pandera or pydantic at every boundary.

☐ Seeds, config and data versions are recorded for every result.

☐ Weighted estimates use the survey weights. Intervals and effect sizes go with every estimate.

☐ Preprocessing sits inside a scikit-learn Pipeline, fitted on training folds only.

☐ The metric matches the decision. Calibration and subgroup performance are checked.

☐ Every experiment is tracked in MLflow.

☐ Tests cover edge cases, including empty inputs and zeros.

☐ Tests use fixtures and parametrize, compare numbers with stated tolerances, isolate I/O, and run with coverage in CI.

☐ Every model has a model card. Every decision has a log entry.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

B10-L1 Target leakage, features that know the answer

B10-L1 exercise 1

Hints. (1) When is the prediction made? (2) Which value is measured after it? (3) March, during the month you predict.

Answer. c) Number of support calls in March Because: r1) March calls happen after the prediction is made, and may be caused by leaving.

Features must come from before the prediction date. March calls are not available on 1 March.

Common wrong answers.

- `a`: Tenure on 1 March is known on 1 March.
- `b`: February is before the prediction date. Safe to use.
- `d`: Plan type on 1 March is known at prediction time.
- `c|r2`: Right answer, wrong reason. February calls are fine. Timing decides.

B10-L1 exercise 2

Hints. (1) Does days_inactive separate the classes perfectly? (2) And contract_months_left, in reverse? (3) An AUC of 0 is as informative as 1.

Answer.

```
1 1.0 1.0
```

Both separate the classes perfectly, one in reverse. A screen must check `max(auc, 1 - auc)`.

Common wrong answers.

- `1.0 0.0`: b is 0.0, a perfect inverse relationship. max(b, 1 - b) turns it into 1.0.

B10-L1 exercise 3

Hints. (1) Everything is judged against the prediction date. (2) Timing, then data-driven checks. (3) Explanation needs a suspect.

Answer.

1. Write down the prediction date and what the model will see then

2. Find the date each feature value becomes known
3. Drop features known only after the prediction date
4. Run the single-feature AUC screen on what remains
5. Explain any high-AUC feature with the data owner, or drop it

Define the moment of prediction, check each feature's timing, then use the screen as a second line of defence.

Common wrong answers.

- Screening first finds the obvious leaks but misses timing leaks with modest AUC. Start from the prediction date.
- Screen first, then explain what the screen flags.

B10-L1 exercise 4

Hints. (1) `X.select_dtypes("number").columns` limits the loop to numeric features. (2) Filter rows with `X[col].notna()` and check `y[known].nunique() == 2`. (3) Flag when `max(auc, 1 - auc) >= threshold`.

Answer.

```python
import pandas as pd
from sklearn.metrics import roc_auc_score


def leakage_screen(X: pd.DataFrame, y: pd.Series, threshold: float = 0.95) -> list[str]:
    """Numeric features that alone separate the classes suspiciously well.

    Args:
        X: Candidate features.
        y: Binary target aligned with X.
        threshold: AUC strength at or above which a feature is flagged.

    Returns:
        Suspect column names, in column order.
    """
    suspects = []
    for col in X.select_dtypes("number").columns:
        known = X[col].notna()
        if known.sum() == 0 or y[known].nunique() < 2:
            continue
        auc = roc_auc_score(y[known], X.loc[known, col])
        if max(auc, 1 - auc) >= threshold:
            suspects.append(col)
    return suspects
```

The screen looks for any single feature that nearly separates the classes. Missingness itself can leak, as `late_only` shows: filled with zeros it would look perfect.

B10-L1 exercise 5

Hints. (1) Would you know it at the moment of the transaction? (2) Can it be recomputed from earlier data? (3) Who fills it in, and when?

Answer.

- Amount: known at transaction time.
- Chargeback: a consequence of fraud, filed weeks later.
- Transaction count: useful, but must count only transactions before this one.
- Reviewer: assigned only to suspected cases, a proxy for the label.

Keep what is known at prediction time, rebuild what can be made point in time, drop the rest.

Common wrong answers.

- chargeback = `keep`: A chargeback is filed after the fraud is found. It is the label in disguise.
- txn_count = `keep`: Computed on the extract date, it includes transactions after this one. Rebuild it point in time.
- reviewer = `keep`: Reviewers are assigned to flagged cases. The feature encodes the labelling process.

B10-L2 Fit on training data only

B10-L2 exercise 1

Hints. (1) Which step computes something from the data? (2) Which uses only a fixed constant? (3) 2.54 cm per inch.

Answer. b) Converting height from inches to centimetres Because: r1) A unit conversion uses a fixed constant. It learns nothing from the rows.

Fixed transformations are safe anywhere. Anything that computes a statistic from rows must be fitted on training rows only.

Common wrong answers.

- `a`: The median is learned from the rows, including test rows.
- `c`: Selection by target correlation learns from test labels. It is the strongest leak of the four.

- `d`: Mean and sd are learned from the rows.
- `b|r2`: Right answer, wrong reason. Timing does not make a step safe. Learning nothing from the data does.

B10-L2 exercise 2

Hints. (1) Did the selection see the rows used for testing? (2) Can it pick features that fit those labels by chance? (3) Far above 0.5.

Answer.

```
0.85
```

With 5,000 features, 20 fit any 100 labels by chance. Selecting them on all rows leaks the test labels. Inside the folds, the same code scores 0.48.

Common wrong answers.

- `0.5`: That is the honest answer for noise. Selection on all rows has seen the test labels, so CV reports 0.85.

B10-L2 exercise 3

Hints. (1) Which rows does the scaler learn from? (2) What must the test set not do? (3) `X_train` and `transform`.

Answer.

```
scaler = StandardScaler().fit(X_train)
X_train_s = scaler.transform(X_train)
X_test_s = scaler.transform(X_test)
```

Fit once on training rows. Transform everything with the same fitted object.

Common wrong answers.

- g2 = `fit_transform`: fit_transform on the test set refits the scaler on test rows.
- g1 = `X`: Fitting on all rows leaks test statistics into training.

B10-L2 exercise 4

Hints. (1) fit_prep: median first, then fill, then mean and std. (2) apply_prep reads only from params. (3) Guard `params["sd"] == 0`.

Answer.

```
import pandas as pd


def fit_prep(train: pd.DataFrame) -> dict[str, float]:
    """Learn imputation and scaling parameters for income from training
    rows.

```

```
    Returns:
        median, mean and sd of income after median imputation.
    """
    median = float(train["income"].median())
    filled = train["income"].fillna(median)
    return {"median": median, "mean": float(filled.mean()), "sd":
    float(filled.std())}


def apply_prep(df: pd.DataFrame, params: dict[str, float]) ->
    pd.DataFrame:
    """Impute and scale income with parameters learned from training
    rows."""
    filled = df["income"].fillna(params["median"])
    if params["sd"] == 0:
        return df.assign(income=0.0)
    return df.assign(income=(filled - params["mean"]) / params["sd"])
```

All learning sits in `fit_prep`. `apply_prep` is a pure function of the data and the learned parameters, so test rows can never influence them.

B10-L2 exercise 5

Hints. (1) Which rows does `both` contain? (2) Whose labels feed the rates? (3) Group the training rows only, and give unseen regions the training mean.

Answer. The bug is on line 6, 7.

Fixed version:

```
import pandas as pd


def encode_region(train: pd.DataFrame, test: pd.DataFrame) ->
    tuple[pd.DataFrame, pd.DataFrame]:

    """Replace region with its training churn rate. Unseen regions get the
    training mean."""
    rates = train.groupby("region")["churned"].mean()
    prior = float(train["churned"].mean())
    return (
        train.assign(region=train["region"].map(rates)),
        test.assign(region=test["region"].map(rates).fillna(prior)),
    )
```

Concatenating before grouping puts test labels into the encoding. Learn rates from training rows, and fall back to the training mean for regions the training data never saw. Inside training, cross-fitted encodings reduce the remaining leak, as scikit-learn's TargetEncoder does.

Common wrong answers.

- Line 8: Mapping both sets with one table is right. The table was built from test labels.

B10-L3 Time-safe features and point-in-time joins

B10-L3 exercise 1

Hints. (1) How will the model be used? (2) Can it see the future in production? (3) Then not in validation either.

Answer. b) Train on months up to a cut-off, test on later months Because: r1) It matches production, where the model predicts a future it has not seen.

A time split reproduces the production situation: learn from the past, predict the future.

Common wrong answers.

- `a`: A random split trains on rows from after the test rows, which production never allows.
- `c`: Backwards in time: the model would learn from the future of its test set.
- `d`: Training accuracy rewards memorising and says nothing about new data.
- `b|r2`: Right answer, wrong reason. The point is mirroring the direction of time, not data quality.

B10-L3 exercise 2

Hints. (1) Is there a score on the prediction date itself? (2) What is the default of allow_exact_matches? (3) True.

Answer.

```
1 9
```

The same-day score matches by default. If it was recorded after the prediction was made, it leaks. Use `allow_exact_matches=False`.

Common wrong answers.

- `3`: allow_exact_matches defaults to True, so the score from the same day matches. Set it to False to exclude it.

B10-L3 exercise 3

Hints. (1) The pandas join for 'last value before'. (2) Exclude exact timestamp matches. (3) `merge_asof` and `False`.

Answer.

```
pd.merge_asof(preds, scores, left_on="as_of", right_on="scored_at",
    by="customer", allow_exact_matches=False)
```

`merge_asof` takes the last earlier match per `by` group.
`allow_exact_matches=False` makes "earlier" strict.

Common wrong answers.

- g1 = `merge`: A plain merge joins every score for the customer, past and future.

B10-L3 exercise 4

Hints. (1) Sort by customer and date first. (2) Running total with groupby(...).cumsum(), minus the current spend. (3) groupby(...).cumcount() counts earlier rows from 0.

Answer.

```
import pandas as pd


def add_prior_spend(orders: pd.DataFrame) -> pd.DataFrame:
    """Spend and order count before each order, per customer.

    Args:
        orders: Columns customer, ordered_at, spend.

    Returns:
        Orders sorted by customer and date, with prior_spend and
    prior_orders.
    """
    o = orders.sort_values(["customer", "ordered_at"])
    return o.assign(
        prior_spend=o.groupby("customer")["spend"].cumsum() - o["spend"],
        prior_orders=o.groupby("customer").cumcount(),
    )
```

Sorting fixes the time order. Subtracting the current spend and counting from 0 make both features strictly prior.

B10-L3 exercise 5

Hints. (1) Does the merge filter scores by date? (2) Which score does tail(1) pick? (3) The latest overall, even from the future. Use merge_asof.

Answer. The bug is on line 6, 7.

Fixed version:

```python
import pandas as pd


def latest_nps(preds: pd.DataFrame, scores: pd.DataFrame) -> pd.DataFrame:
    """Most recent NPS strictly before each prediction date."""
    joined = pd.merge_asof(
        preds.sort_values("as_of"),
        scores.sort_values("scored_at"),
        left_on="as_of",
        right_on="scored_at",
        by="customer",
        allow_exact_matches=False,
    )
    return joined[["customer", "as_of", "nps"]].sort_values(["customer", "as_of"]).reset_index(drop=True)
```

A merge on customer followed by "latest" picks scores from after the prediction date. `merge_asof` with `allow_exact_matches=False` picks the last strictly earlier score.

Common wrong answers.

- Line 8: Selecting and sorting columns is fine. The rows chosen before it include future scores.